

AutoMapper in C# with Examples

In this article, I will discuss **AutoMapper in C#** with Examples. At the end of this article, you will have a very good understanding of the following pointers.

1. **Why do we need AutoMapper in C#?**
2. **Mapping Object in Traditional Approach in C#**
3. **What is AutoMapper in C#?**
4. **How to use AutoMapper in C#?**
5. **What will happen if the Source and Destination Property Names are Different?**
6. **How do Map Properties when the Property Names are Different using AutoMapper?**
7. **Advantages and Disadvantages of using AutoMapper in C#**
8. **When to use AutoMapper in C#?**

Why do we need AutoMapper in C#?

Let's understand why we need Automapper in C# with an example. Let's say we have the following two classes: **Employee** and **EmployeeDTO**. First, create a class file named **Employee.cs**, and then copy and paste the following code. This is a very simple class having 4 properties.

```
namespace AutoMapperDemo
{
    public class Employee
    {
        public string Name { get; set; }
        public int Salary { get; set; }
        public string Address { get; set; }
        public string Department { get; set; }
    }
}
```

Next, create another class file with the name EmployeeDTO.cs and then copy and paste the following code into it. This class is identical to the Employee class, i.e., having 4 properties.

```
namespace AutoMapperDemo
{
    public class EmployeeDTO
    {
        public string Name { get; set; }
        public int Salary { get; set; }
        public string Address { get; set; }
        public string Department { get; set; }
    }
}
```

Now, what is our business requirement is to copy the data or transfer the data from the **Employee** object to the **EmployeeDTO** object. In the traditional approach (without using Automapper), first, we need to create and populate the Employee object, as shown in the image below.

```
//Create and Populate Employee Object
Employee emp = new Employee
{
    Name = "James",
    Salary = 20000,
    Address = "London",
    Department = "IT"
};
```

Once you have the employee object, you need to create the **EmployeeDTO** object and copy the data from the Employee object to the EmployeeDTO object, as shown in the image below.

```
//Mapping Employee Object to EmployeeDTO Object
EmployeeDTO empDTO = new EmployeeDTO
{
    Name = emp.Name,
    Salary = emp.Salary,
    Address = emp.Address,
    Department = emp.Department
};
```

Mapping Object in Traditional Approach in C#

Let us understand how the Object is Mapped to another Object in C# using the Traditional Approach. For a better understanding, please have a look at the following example. First, we create an instance of the Employee object and populate the four properties with the required data. Then, we create an instance of the EmployeeDTO class and populate the EmployeeDTO properties with the values from the Employee object. Finally, we displayed the values of the EmployeeDTO object.

```
using System;
namespace AutoMapperDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create and Populate Employee Object
            Employee emp = new Employee
            {
                Name = "James",
                Salary = 20000,
                Address = "London",
                Department = "IT"
            }
        }
    }
}
```

```

    };
    //Mapping Employee Object to EmployeeDTO Object
    EmployeeDTO empDTO = new EmployeeDTO
    {
        Name = emp.Name,
        Salary = emp.Salary,
        Address = emp.Address,
        Department = emp.Department
    };
    Console.WriteLine("Name:" + empDTO.Name + ", Salary:" + empDTO.Salary + ", Address:" + empDTO.Address + ", Department:" + empDTO.Department);
    Console.ReadLine();
}
}
}

```

If you run the application, you will get the output as expected. But tomorrow, what will you do if the data, i.e., the properties in the Employee and EmployeeDTO classes, are increased? Then, you must write the data moving code for each property from the source class to the destination class. That means the code mapping is repeated between the source and the destination. Again, if the same mapping is at different places, then you need to make the changes at different places, which is time-consuming and error-prone.

In Real-Time Projects, we often need to map the objects between the UI Layer or Presentation Layer and Business Logic layers. Mapping the objects between them is very hectic using the abovementioned traditional approach. So, is there any simplest solution by which we can map two objects? Yes, there is, and the solution is **AutoMapper**.

What is AutoMapper in C#?

AutoMapper is a popular open-source library in C# that simplifies mapping data between different classes or objects. It helps eliminate repetitive and error-prone code when copying data from one object to another. AutoMapper is especially useful in scenarios like mapping database entities to DTOs (Data Transfer Objects) or ViewModel objects.

The AutoMapper in C# is a mapper between two objects. That is, AutoMapper is an Object-Object Mapper. It maps the properties of two different objects by transforming the input object of one type to the output object of another.

How to use AutoMapper in C#?

Let us understand how to use AutoMapper in C# with an example. We will map the same Employee Object with the EmployeeDTO Object we discussed in the first example. We need to map each **Employee Property** to the corresponding **EmployeeDTO Property** using AutoMapper, as shown in the image below.

```

public class Employee
{
    public string Name { get; set; }
    public int Salary { get; set; }
    public string Address { get; set; }
    public string Department { get; set; }
}

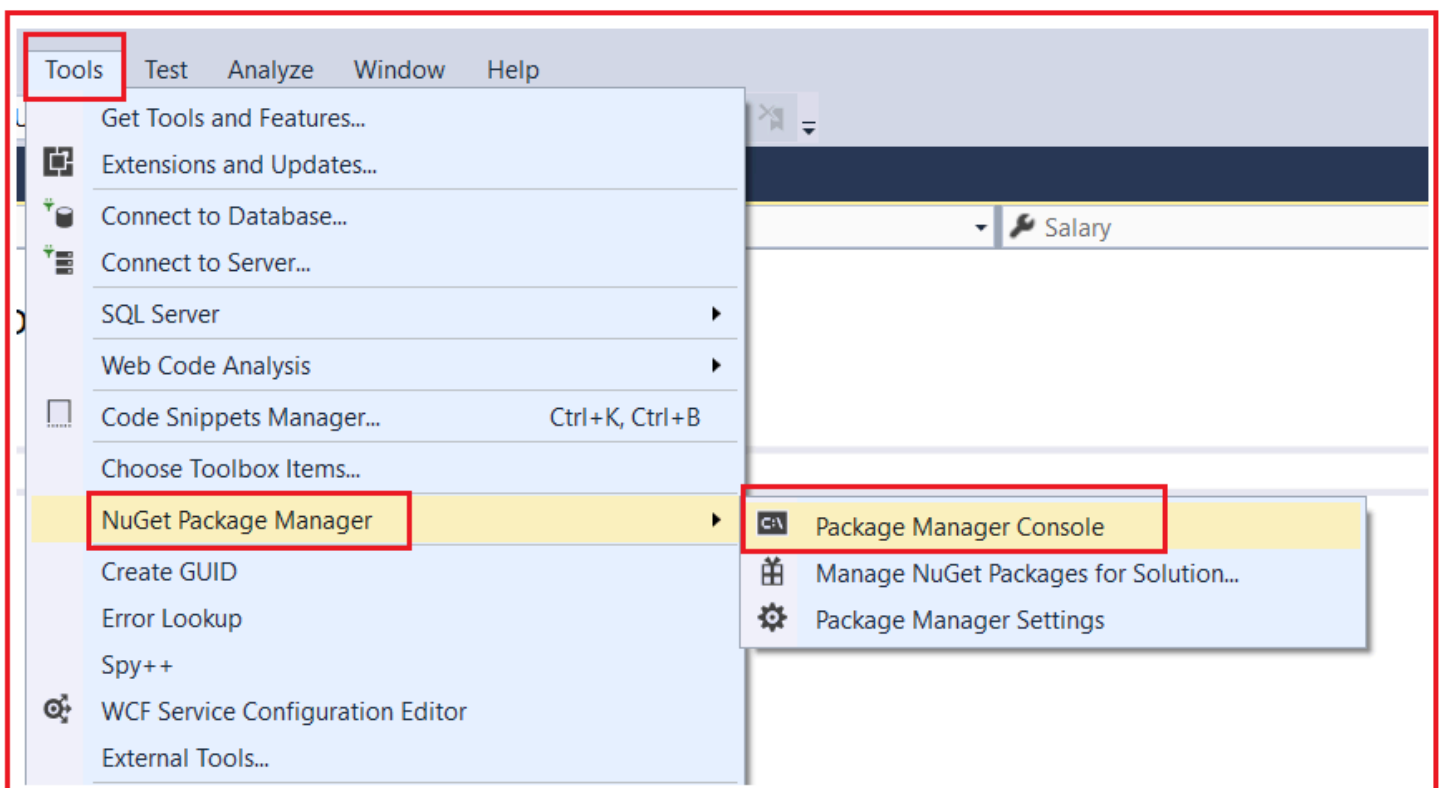
public class EmployeeDTO
{
    public string Name { get; set; }
    public int Salary { get; set; }
    public string Address { get; set; }
    public string Department { get; set; }
}

```

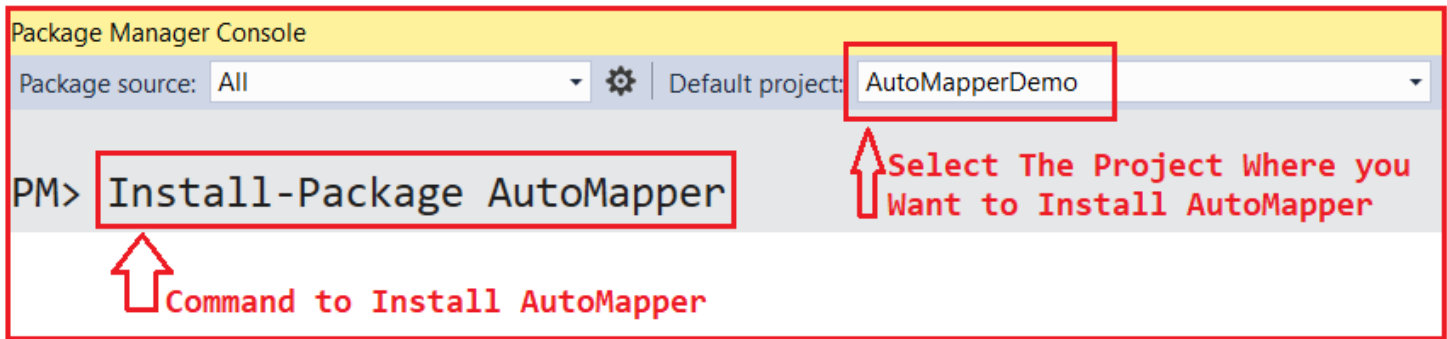
Let's discuss the step-by-step procedure to use AutoMapper in C#.

Step 1: Installing AutoMapper Library in Your Project

AutoMapper is an open-source library present in [GitHub](#). To install this library, open the Package Manager Console window. To Open the Package Manager Console window, select Tools => NuGet Package Manager => Package Manager Console from the context menu, as shown in the image below.

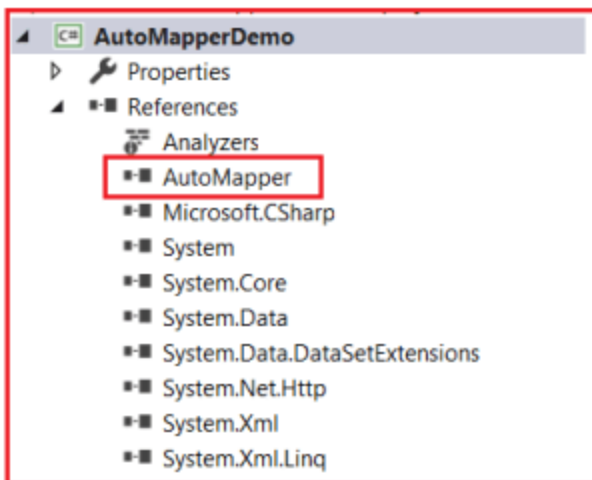


Once you open the Package Manager Console window, type the command **Install-Package AutoMapper** and press the enter key to install the AutoMapper Library in your project, as shown in the image below.



Note: If you are getting any errors while installing AutoMapper, try to install the lower version of AutoMapper, which is compatible with your .NET Framework Version. I am working with .NET Framework 4.8 and installing AutoMapper 10.1.1 version, which is compatible with .NET Framework 4.8.

Once you install the **AutoMapper** Library in your project, it will add a reference to the **AutoMapper.DLL**, which you can find in the project references section, as shown in the image below.



Step2: Configuring and Initializing AutoMapper in C#

In the second step, we must configure and Initialize the AutoMapper for our project. Once we have defined the types (i.e., classes to be mapped by AutoMapper, in our example, once we define the Employee and EmployeeDTO classes), then we need to configure the Mapping for the two types using the constructor of the **MapperConfiguration** class.

You need to remember that you can create only one **MapperConfiguration Instance Per AppDomain**, and it should be instantiated during the application Start-Up. The syntax to create the MapperConfiguration instance is given below. Here, we need to provide the Mapper Configuration using Lambda Expression and tell the Source and Destination types using the generic CreateMap method.

```
var config = new MapperConfiguration(cfg =>
    cfg.CreateMap<TSource, TDestination>()
);
```

The type on the left-hand side is the source type, i.e., **TSource**. In our example, it will go to the **Employee** object, and the type on the right-hand side is the destination type, i.e., **TDestination**. In our

example, it will be going to the **EmployeeDTO** object. So, to map the **Employee** with the **EmployeeDTO** object, you need to create the mapper configuration instance as follows.

```
var config = new MapperConfiguration(cfg =>
    cfg.CreateMap<Employee, EmployeeDTO>()
);
```

Once we create the MapperConfiguration instance where we configure the Mappings, then we need to initialize the Mapper. To Initialize the Mapper, what we need to do is, we need to pass the MapperConfiguration to the constructor of the Mapper class, as shown in the image below.

```
var config = new MapperConfiguration(cfg => {
    cfg.CreateMap<Employee, EmployeeDTO>();
});

var mapper = new Mapper(config);
```

So, what we are going to do is we will create a separate class where we will do all our mapping configurations. So, create a class file named **MapperConfig.cs** and copy and paste the following code. In the below class, we have one method where we configure the required mappings and initialize the Mapper. We are also returning the Mapper instance, using which we will do the mapping. Here, we use the CreateMap generic method and specify the Source and Destination Types. Here, the Source Type is Employee, and the Destination Type is EmployeeDTO, i.e., we will map the Employee Object with the EmployeeDTO object using AutoMapper.

```
using AutoMapper;
namespace AutoMapperDemo
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {
            //Provide all the Mapping Configuration
            var config = new MapperConfiguration(cfg =>
            {
                //Configuring Employee and EmployeeDTO
                cfg.CreateMap<Employee, EmployeeDTO>();
                //Any Other Mapping Configuration ....
            });
            //Create an Instance of Mapper and return that Instance
            var mapper = new Mapper(config);
            return mapper;
        }
    }
}
```

```
}  
}
```

Step 3: Using AutoMapper in C#:

Once we have Configured the Mapping Configurations, i.e., configuring the Source and Destination Types, we can use that Mapping Configuration to Map the Source and Destination Objects. We need to use the Mapper instance created in the previous step. So, we need to call the InitializeAutomapper static method using the MapperConfig class name, which will return the Mapper instance. Once we have the Mapper instance, then using the Map method of the Mapper instance, we can provide the Source and Destination objects in two ways, which are shown in the image below.

```
//Create and Populate Employee Object i.e. Source Object  
Employee emp = new Employee[...];  
  
//Initializing AutoMapper  
var mapper = MapperConfig.InitializeAutomapper();  
  
//Way1: Specify the Destination Type and to The Map Method pass the Source Object  
//Now, empDT01 object will having the same values as emp object  
var empDT01 = mapper.Map<EmployeeDTO>(emp);  
  
//Way2: Specify the both Source and Destination Type  
//and to The Map Method pass the Source Object  
//Now, empDT02 object will having the same values as emp object  
var empDT02 = mapper.Map<Employee, EmployeeDTO>(emp);
```

For a better understanding, modify the Main method of the Program class as follows. Here, we are Mapping the Source Employee Object with the Destination EmployeeDTO object using AutoMapper in C#. The following example code is self-explained.

```
using System;  
namespace AutoMapperDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            //Create and Populate Employee Object i.e. Source Object  
            Employee emp = new Employee  
            {  
                Name = "James",  
                Salary = 20000,  
                Address = "London",  
                Department = "IT"  
            };  
  
            //Initializing AutoMapper  
            var mapper = MapperConfig.InitializeAutomapper();
```

```

Object        //Way1: Specify the Destination Type and to The Map Method pass the
               //Now, empDTO1 object will having the same values as emp object
               var empDTO1 = mapper.Map<EmployeeDTO>(emp);

               //Way2: Specify the both Source and Destination Type
               //and to The Map Method pass the Source Object
               //Now, empDTO2 object will having the same values as emp object
               var empDTO2 = mapper.Map<Employee, EmployeeDTO>(emp);
               Console.WriteLine("Name: " + empDTO1.Name + ", Salary: " + empDTO1
", Address: " + empDTO1.Address + ", Department: " + empDTO1.Department);
               Console.ReadLine();
           }
       }
}

```

Output: **Name: James, Salary: 20000, Address: London, Department: IT**

What will happen if the Source and Destination Property Names are Different?

Let us understand this with an example. Let us change the property names of the Destination class. Let's change the **EmployeeDTO** class property names. Change the **Name** property to **FullName** and **Department** property to **Dept**. So, modify the EmployeeDTO class as follows.

```

namespace AutoMapperDemo
{
    public class EmployeeDTO
    {
        public string FullName { get; set; }
        public int Salary { get; set; }
        public string Address { get; set; }
        public string Dept { get; set; }
    }
}

```

Next, modify the Main method of the Program class as follows to use the new property names while displaying the values.

```

using System;
namespace AutoMapperDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create and Populate Employee Object i.e. Source Object
            Employee emp = new Employee
            {
                Name = "James",
                Salary = 20000,
            }
        }
    }
}

```

```

        Address = "London",
        Department = "IT"
    };

    //Initializing AutoMapper
    var mapper = MapperConfig.InitializeAutomapper();
    //Way1: Specify the Destination Type and to The Map Method pass the
Object
    //Now, empDTO1 object will having the same values as emp object
    var empDTO1 = mapper.Map<EmployeeDTO>(emp);

    //Way2: Specify the both Source and Destination Type
    //and to The Map Method pass the Source Object
    //Now, empDTO2 object will having the same values as emp object
    var empDTO2 = mapper.Map<Employee, EmployeeDTO>(emp);
    Console.WriteLine("Name:" + empDTO1.FullName + ", Salary:" + empD1
", Address:" + empDTO1.Address + ", Department:" + empDTO1.Dept);
    Console.ReadLine();
}
}
}

```

With the above changes in place, now run the application, and you should get the following output.

```
Name:, Salary:20000, Address:London, Department:
```

The above output clearly shows that the **Name** and **Department** are empty, meaning these two properties are not mapped from the Source to the Destination types. So, the point that you need to remember is when the property names are different in Source and Destination types, then by default, AutoMapper will not map those properties from the source object to the destination object.

How do Map Properties when the Property Names are Different using AutoMapper?

If the Property names differ in Source and Destination types, then you can map such properties in AutoMapper using the **ForMember** option. So, to Map the **Name** property of the Source Object with the **FullName** property of the Destination Object and the **Department** property of the Source Object with the **Dept** property of the Destination Object, we need to provide mapping configuration for these two properties in the mapping configuration. So, modify the MapperConfig class file to provide such property mapping configurations. In our upcoming articles, we will discuss the ForMember and MapForm options in detail.

```

using AutoMapper;
namespace AutoMapperDemo
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {
            //Provide all the Mapping Configuration
            var config = new MapperConfiguration(cfg =>

```

```

{
    //Configuring Employee and EmployeeDTO
    cfg.CreateMap<Employee, EmployeeDTO>()
    //Provide Mapping Configuration of FullName and Name Property
    .ForMember(dest => dest.FullName, act => act.MapFrom(src => sr

    //Provide Mapping Dept of FullName and Department Property
    .ForMember(dest => dest.Dept, act => act.MapFrom(src => src.De
    //Any Other Mapping Configuration ....
});
//Create an Instance of Mapper and return that Instance
var mapper = new Mapper(config);
return mapper;
}
}
}

```

With the above changes in place, now run the application, and you should see the output as expected.

Advantages and Disadvantages of using AutoMapper in C#

Using AutoMapper in C# can offer several advantages and disadvantages. It's important to consider both sides to determine whether it's the right choice for your scenario.

Advantages of using AutoMapper:

- **Reduces Repetitive Code:** AutoMapper eliminates the need to manually write repetitive code (like property-to-property assignment) to map properties from one object to another. This can significantly reduce the amount of code you have to maintain.
- **Improves Code Readability and Maintainability:** With less code to manage for object mapping, your codebase becomes cleaner and easier to maintain.
- **Consistency in Mapping:** AutoMapper ensures consistent mapping rules across the application, reducing the likelihood of errors that can occur in manual mappings.
- **Custom Mapping Capabilities:** While it automates simple mappings, AutoMapper also allows for custom configuration for complex mappings, giving you flexibility when needed.
- **Ease of Use:** AutoMapper is generally easy to set up and use, especially for straightforward mappings, making it a convenient tool for rapid development.
- **Reduces Human Error:** Manual object mapping is prone to errors, especially in large and complex projects. AutoMapper automates this process, thereby reducing the chance of mistakes.

Disadvantages and considerations of using AutoMapper:

- **Performance Overhead:** AutoMapper can introduce performance overhead, especially if not configured properly. This can be a concern in high-performance or resource-limited environments.
- **Complexity in Debugging:** Debugging can become more challenging with AutoMapper, particularly when dealing with complex mappings or when the mapping configuration is not straightforward.
- **Learning Curve:** While AutoMapper is easy to use for basic scenarios, mastering its advanced features and best practices can require a significant time investment.

- **Overreliance on Convention:** AutoMapper relies heavily on naming conventions. You must write more custom configurations if your source and destination objects don't follow a consistent naming convention.
- **Unnecessary Complexity for Simple Tasks:** For very simple object mapping scenarios, using AutoMapper might be overkill, and manual mapping could be more straightforward and efficient.

When to use AutoMapper in C#?

You should consider using AutoMapper in C# under the following circumstances:

- **Large Projects with Complex Object Models:** In large-scale applications with complex domain models and Data Transfer Objects (DTOs), AutoMapper can save significant time and effort by automating the mapping process.
- **Projects with Repetitive Mapping Code:** If your project involves a lot of repetitive and tedious mapping code (like copying properties from one object to another), AutoMapper can help reduce this boilerplate code, making your codebase cleaner and easier to maintain.
- **When Consistency in Mapping Rules is Important:** AutoMapper helps maintain consistent mapping rules across your application. This is particularly useful in team environments or large projects where different developers might otherwise implement mappings inconsistently.
- **CRUD Operations in Layered Architecture:** In applications with a clear separation of concerns, such as MVC applications, AutoMapper is valuable for transforming data between layers (e.g., from data access objects to business logic objects or business logic objects to view models).
- **When You Need Customizable Mapping Logic:** AutoMapper is not just for straightforward mappings; it also allows for complex and custom mapping logic. This is useful when you have specific rules for how certain properties should be mapped or transformed.
- **In Applications Where Development Speed is a Priority:** Rapid application development can benefit from AutoMapper, as it speeds up writing and updating code involving object mappings.

However, there are scenarios where AutoMapper may not be the best choice:

- **For Simple Projects:** If your project is simple, with few mappings, or if the mappings are straightforward, the overhead of introducing a third-party library might not be justified.
- **Performance-Critical Applications:** Be cautious in performance-sensitive applications, as AutoMapper can introduce some overhead, especially if not configured optimally.
- **When Explicit Control is Preferred:** In cases where you need or prefer explicit control over every aspect of the mapping, or if you want to avoid the abstraction introduced by AutoMapper, manual mapping might be a better choice.

In the next article, I will discuss [Automapper Complex Mapping in C#](#) with Examples. In this article, I try to explain **AutoMapper in C#** with Examples. I hope you enjoy this AutoMapper in C# with Examples article.